IOT104TC PROJECT

Smart Classroom

Energy-Saving, Safety & Asset Monitoring System

Local-first classroom control with real sensors, actuators, MQTT operations, dashboard analytics and AI-assisted commands.

UNO Edge Node

Sensors, state machine and actuators

UART + ACK

ESP32 Gateway

Wi-Fi, MQTT and dashboard bridge

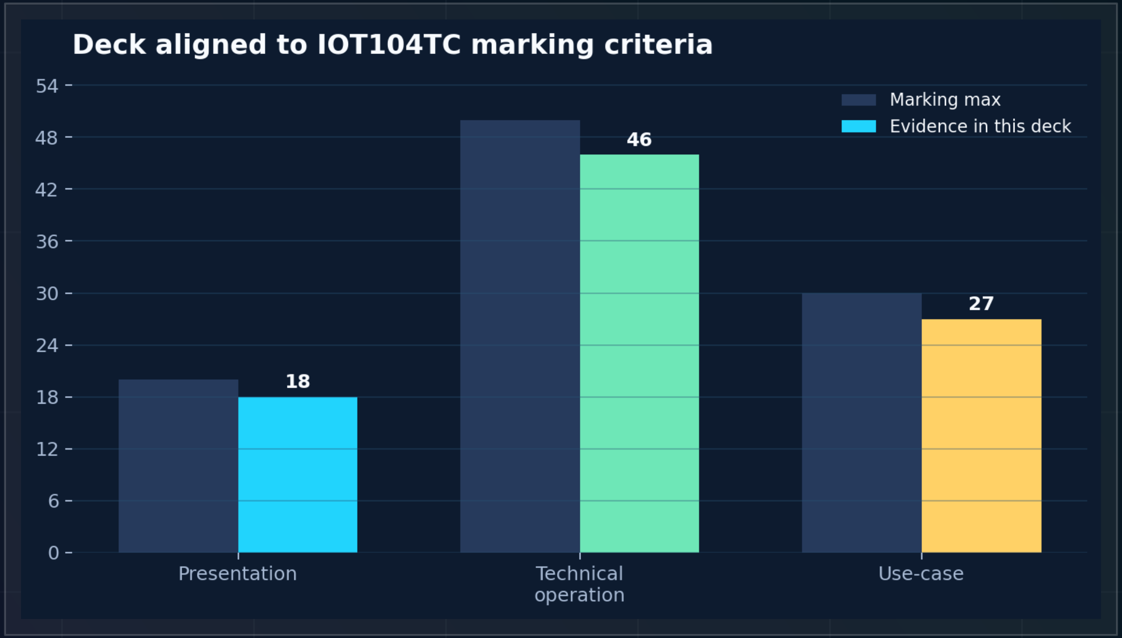
9+	**5**	**1s**	**432m**
hardware signals	priority modes	telemetry cadence	recorded evidence

Local safety works even when the network is offline.

ASSESSMENT FIT

Built around the marking scheme

The demo connects hardware, software, architecture, security/privacy, operations and a practical classroom use-case.



Application

Energy, comfort and safety in one classroom service.

Architecture

UNO local control. ESP32 MQTT gateway.

Evidence

Telemetry, screenshots and self-check.

Security

Fail-safe local control and ignored secrets.

USE-CASE

A classroom that reacts locally and reports globally

The system behaves like a small building-management service, not a sensor list.

01
Sense
Occupancy, light, temperature, gas, flame and emergency button.

->

02
Decide
Priority state machine: safety first, then comfort and energy saving.

->

03
Act
Lamp, fan, PWM, tach, buzzer and status light.

->

04
Operate
MQTT dashboard, manual override, presets, charts and AI timeline.

Safety
highest priority

Comfort
adaptive fan/lamp

Ops
ACK + persistence

Data
charts + history

Real-time control stays on UNO

Networking adds visibility and commands, but does not become a dependency for safety.

Input

Sensors

- D2 PIR / A0 Light
- A1 Gas / A2 LM35
- D8/A3 Flame
- D4 Button

—>

Edge

Arduino UNO

- millis loop
- state machine
- relay / PWM / tach
- buzzer + status light

—>

Gateway

ESP32

- Serial2 UART
- Wi-Fi reconnect
- MQTT publish
- command bridge

—>

Service

Laptop

- Mosquitto
- web dashboard
- history store
- AI planner

Design decision: ESP32 is a gateway; UNO remains the controller of record.

Dense hardware, clean responsibility

The build is complex, but each module has a clear role in the service.

48-kit modules

#18 PIR
#31 TEMT6000
#25 Gas Sensor
#16 Flame Sensor
#5 Active Buzzer
Traffic Light Module

External components

UNO R3
ESP32 Gateway
LM35 powered from ESP32 5V
6-channel relay
12V 4-wire fan
LED + resistor

Wiring rules

Common ground is required.
UNO A5 to ESP32 RX2 uses divider.
Relay logic uses RELAY_ON / RELAY_OFF.
No capacitor is added in this build.

ENGINEERING FIX

LM35 became stable after moving power to ESP32 5V and adding ADC guard compensation.

Priority is a state machine

Mixed behavior stays predictable because safety, sensor health, manual control, cooling, lighting and saving are ordered.

SAFETY_ALARM	gas/flame/button
SENSOR_ERROR	invalid sensor state
MANUAL	operator override
COOLING	temperature high
LIGHTING	occupied + dark
ENERGY_SAVING	empty or bright
NORMAL	monitoring

WHY IT MATTERS

Dashboard explains what happened and why.

That turns raw telemetry into an operational classroom service.

The highest-priority path is visible and local

The button gives a safe live demo while gas and flame remain the real sensor paths.

Trigger

Three inputs

gasDanger
flameDetected
demoEmergency

>

Decision

SAFETY ALARM

Overrides lighting, cooling and energy-saving logic immediately.

>

Action

Physical response

red light
pulsed buzzer
fan relay ON
PWM = 255

Demo first: press D4, then watch red light, buzzer, fan and dashboard alarm.

Human-friendly automation

The classroom avoids waste without making the controls feel abrupt.

Lighting

Bright room: lamp stays off even if PIR sees motion.
Dark room + occupied: lamp turns on.
No occupancy timeout: lamp turns off.

Cooling

Temperature threshold starts the relay.
PWM ramps fan intensity.
Tach RPM is interrupt-based and never blocks the system.

10s
occupancy timeout

PWM
smooth fan path

ACK
config confirmed

Local
works offline

Telemetry goes up; commands come back with ACK

The protocol is simple enough to debug and structured enough for a dashboard.

```
SC2,mode=SAFETY_ALARM,pir=1,temp10=286,gas=520
fan=1,pwm=255,rpm=1320,status=R
```

UNO
Emit
SoftwareSerial line every second

—>

ESP32
Parse
Serial2 + Wi-Fi reconnect

—>

MQTT
Publish
JSON plus key topics

—>

Web
Operate
Config + presets manual mode

ACK loop proves the command actually landed on the UNO.

An operator console, not a raw JSON page

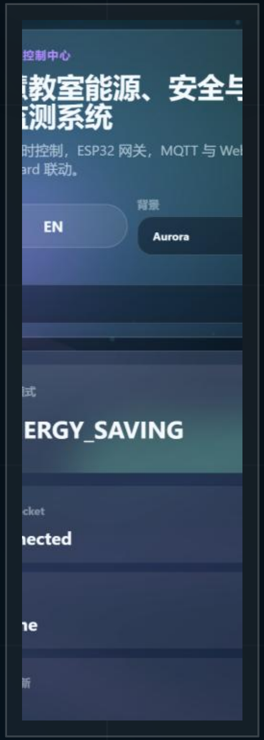
Large cards, live charts, mobile access, manual mode and scene presets make the system usable.

Auto

manual switch

Scenes

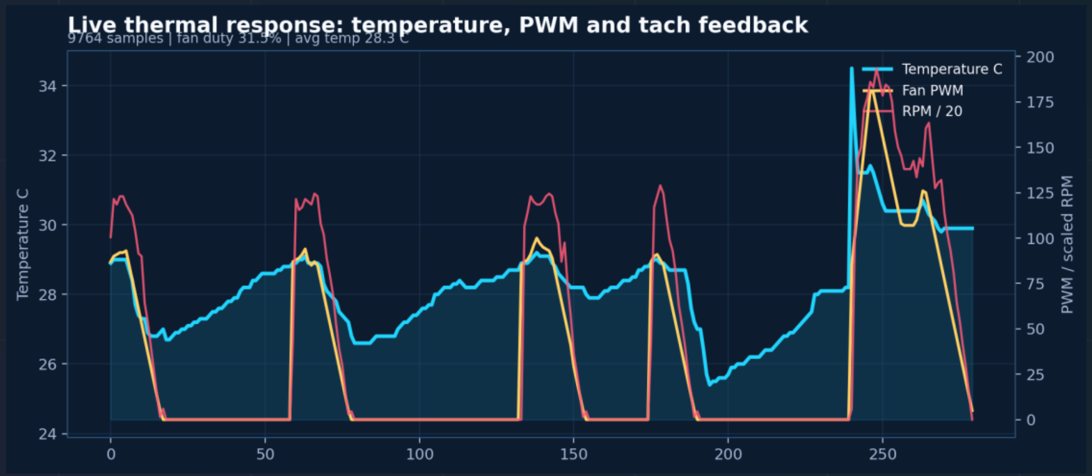
one-click presets

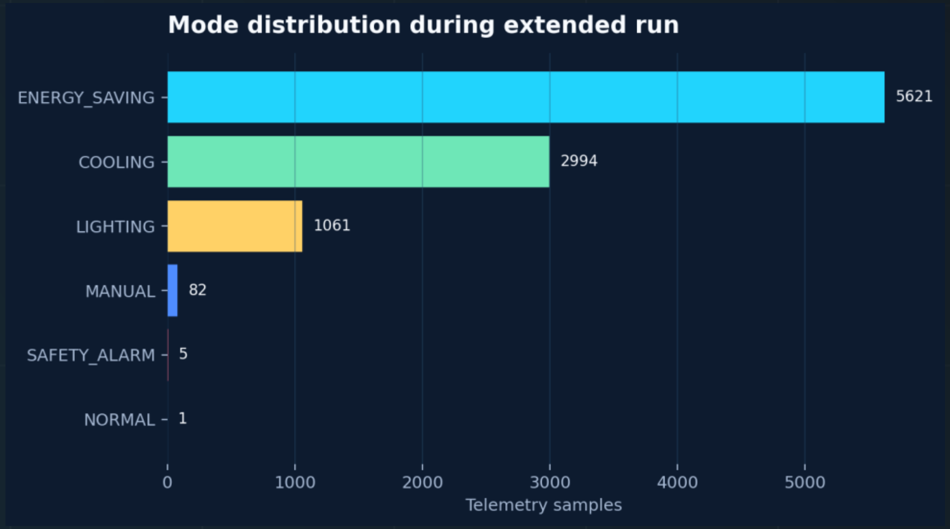


Dashboard screenshots captured from the local running system.

The demo has history, not just live values

Recorded data supports behavior explanation: modes, temperature response, fan duty and safety events.

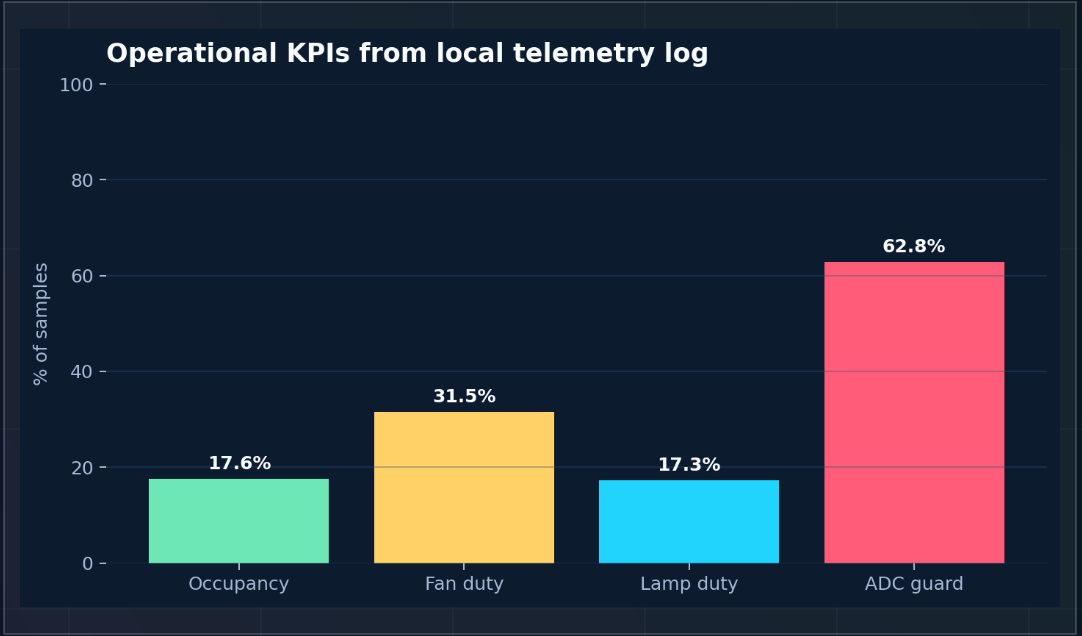




9,764 samples | avg temp 28.3C | fan duty 31.5% | safety events 5

Complexity comes from closed loops

Sensors, control, gateway, dashboard, persistence and AI all interact with the same physical system.



9,764
telemetry samples

12+
MQTT topics

2-way
UART command + ACK

Local-first
safety path offline

34/34
delivery self-check PASS

AI OPERATIONS LAYER

Natural language becomes structured control

AI is useful because it turns vague operator intent into ordered tasks and timed actions.

Voice

Capture

Browser mic input

- >

STT

Text

faster-whisper

- >

Qwen

Plan

structured tasks

Timeline

Delayed actions auto-clear after completion.

Data

Overrides guide comfort.

Demo-friendly, but risk-aware

The project separates safety-critical local control from network convenience and AI assistance.

Safety boundary

Safety alarm, buzzer, red light and fan response are local on UNO.

Privacy boundary

Microphone is user-triggered and requires browser permission.

Operations boundary

MQTT status, ACK logs, persistence and self-check reduce demo risk.

Production hardening path: authenticated MQTT, TLS, enclosure, fuse, isolation and power filtering.

Show the strongest engineering first

The audience should see safety and edge control before dashboard polish and AI features.

P1
Safety
button -> red light -> buzzer -> fan high

->

P2
Edge
UNO local controller and priority modes

->

P3
Energy
PIR + light + temp drive lamp and fan

->

P4
IoT
UART, MQTT topics and ACK command loop

->

P5
AI
voice, Qwen, timeline and data preference

Do not open with AI. Open with physical safety response.

Ready to inspect, rebuild and present

The final package includes sources, assembly materials, scripts and a repeatable delivery check.

References

Arduino UNO R3 docs
Espressif Arduino-ESP32 Serial docs
Eclipse Mosquitto config docs
Arduino PubSubClient docs
Texas Instruments LM35 page

Self-check gates

PPT structure
required docs
firmware presence
secret scan
Node/Python syntax
UNO + ESP32 compile

Final claim: a local-first IoT classroom controller with safety, energy saving, operations and evidence-backed intelligence.